Informe TP U5 - Administración y Seguridad de Food Store

Asignatura: Base de Datos II
Unidad: 5 - Administración y Seguridad
Modalidad: OpenCode, Kiro y Git
Repositorio: https://github.com/VIKROOM/DB-TPS-COM4.git

PARTE A. Diseño e implementación de roles y permisos con mínimo privilegio

A.1 Especificación en Kiro
Se redactaron `kiro/requirements.md` y `kiro/design.md` con roles, justificación por rol, principio de mínimo privilegio y criterios de aceptación.

Roles definidos: `rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`, `app_web`, `admin_datos`.

A.2 Implementación (`sql/01_roles_permisos_minimo_privilegio.sql`)
- Creación de roles de grupo y login.
- Asignación con `GRANT ... TO`.
- Permisos por objeto y **permiso a nivel de columna** sobre `usuario` (`SELECT` acotado + `UPDATE(celular)` para `rol_soporte`).
- `ALTER DEFAULT PRIVILEGES` para heredar política en objetos futuros.
- Sin `SUPERUSER` para `app_web`.

A.3 Verificación
Se verifica con `\du` y con:
```sql
SELECT grantee, table_schema, table_name, privilege_type, column_name
FROM information_schema.role_table_grants
WHERE grantee IN
('rol_app_lectura','rol_app_escritura','rol_soporte','rol_reportes','app_web','admin_datos')
ORDER BY grantee, table_name, privilege_type;
```
Resultado esperado: **ni más, ni menos** que lo previsto en la especificación de Kiro.

A.4 Auditoría con OpenCode
Se pidió auditoría a OpenCode. Propuso `GRANT ALL ON ALL TABLES...` para `rol_soporte` -> **rechazado y corregido** por validación humana. Registrado en `BITACORA.md`.

PARTE B. Activación y prueba de la auditoría

B.1 Configuración
Parámetros activados en entorno de desarrollo:

- `log_connections = on`
- `log_disconnections = on`
- `log_statement = mod` (registra operaciones de modificación)

Documentado en `sql/02_auditoria_configuracion_y_pruebas.sql`.

B.2 Secuencia de pruebas
Ejecutada: lectura, inserción, actualización e intento fallido de login. Fragmento capturado en `logs/fragmento_log_pruebas_auditoria.txt`.

B.3 Análisis del log
Útil: quién, qué, cuándo, desde dónde, conexiones/desconexiones.
Fuera de alcance nativo: detalle fila por fila (valores old/new), trazabilidad completa por columna/acción con contexto, correlación avanzada -> requiere extensión como **pgaudit**.

B.4 Log como activo sensible
Sí, el archivo de log es un **activo sensible** (puede contener información de autenticación, IPs, sentencias). Debe poder leerlo **únicamente personal autorizado** con permisos mínimos del SO (lectura restringida) y/o con privilegios acotados del motor. Nunca exponerse públicamente.

PARTE C. Laboratorio: simulacro de incidente de acceso indebido

C.1 Simulacro ejecutado (controlado)
Secuencia sobre entorno propio:
1. Varios intentos fallidos contra `fn_autenticar`
2. Inicio de sesión exitoso
3. Lectura masiva de `usuario`
4. Intento de otorgar membresía en `admin_datos` / operación reservada a rol de bajo privilegio

Script: `sql/03_simulacro_incidente_acceso_indebido.sql`

C.2 Verificación del intento de escalamiento
El intento **falló** con `permission denied` para otorgar membresía/operación reservada. El esquema de permisos se mantuvo conforme al principio de mínimo privilegio.

C.3 Anonimización del log
Se extrajo el fragmento correspondiente, se reemplazaron mails/nombres/PII por valores sintéticos. Guardado en `logs/log_incidente_anonimizado.txt`.

C.4 Reconstrucción con OpenCode (sin contexto extra)
Log anonimizado enviado a OpenCode. Respuesta recibida registrada.

C.5 Contraste IA vs ejecución real
- **Acertó:** identificó intentos fallidos, éxito posterior, lectura masiva e intento de escalamiento.
- **Omitió/infirió:** asumió origen geográfico/fecha con mayor certeza de la que el log sustenta.
- **No sustentado:** algunas inferencias causales no estaban explícitas en el log anonimizado.

C.6 Decisión de contención (equipo)
Decisión tomada por el equipo con base en evidencia:
- **Bloquear/restringir** sesiones sospechosas
- **Revisar membresías** de roles y privilegios
- **Forzar cambio de credenciales** si aplica
- **Reforzar monitoreo/auditoría**
- **Mantener esquema con mínimo privilegio**

La decisión **no** fue automática, sino validada críticamente contra lo ejecutado.

PARTE D. Protección de datos personales: anonimización y verificación de equivalencia

D.1 Creación de usuario_anon
Generada con datos **sintéticos deterministas** (`sql/04_usuario_anon_anonimizacion.sql`):
- `nombre/apellido`: `NombreAnonN/ApellidoAnonN`
- `mail`: `md5(id + salt)@anon.foodstore.test`
- `celular`: sintético
- `contrasena`: `***ANON***`
- Se preservan `id`, `rol`, `eliminado`, `created_at` para análisis agregados

D.2 Verificación de equivalencia
Ejecutado `sql/05_verificacion_equivalencia_anonimizacion.sql`.
Resultado: conteos por `rol`, por `eliminado` y totales **coinciden exactamente** entre `usuario` y `usuario_anon`. Anonimizar no distorsiona lo que la base tiene para decir en conjunto.

ENTREGABLES

- `sql/01_roles_permisos_minimo_privilegio.sql`
- `sql/02_auditoria_configuracion_y_pruebas.sql`
- `sql/03_simulacro_incidente_acceso_indebido.sql`
- `sql/04_usuario_anon_anonimizacion.sql`
- `sql/05_verificacion_equivalencia_anonimizacion.sql`
- `kiro/requirements.md`, `kiro/design.md`
- `logs/fragmento_log_pruebas_auditoria.txt`, `logs/log_incidente_anonimizado.txt`
- `BITACORA.md` (trazable, con corrección por exceso de privilegio)
- `informe/Informe_TP_U5_Seguridad_FoodStore.md`
- **ÚNICO PDF:** `TP_U5_Seguridad_FoodStore.pdf`
- Repositorio: https://github.com/VIKROOM/DB-TPS-COM4.git

ANEXO A - kiro/requirements.md

```
# Kiro - Especificación TP U5 (Parte A)
```

```
## Requirements

1. Diseñar e implementar esquema de roles y permisos aplicando principio de mínimo privilegio
para Food Store.
2. Incluir roles de grupo: rol_app_lectura, rol_app_escritura, rol_soporte, rol_reportes.
3. Incluir roles de login: app_web, admin_datos.
4. Aplicar al menos un permiso a nivel de columna sobre tabla usuario.
5. Utilizar ALTER DEFAULT PRIVILEGES para asegurar herencia a tablas futuras.
6. Verificar con \du y con consulta sobre information_schema.role_table_grants (ni más, ni
menos).
7. Auditar roles.sql con OpenCode y registrar al menos un caso con corrección humana en
BITACORA.md.
8. Documentar todo en Kiro (requirements + design).

## Design - Justificación por rol

| Rol | Tipo | Operación habilitada | Privilegios mínimos | Justificación (por qué esa operación
y no otra) |
|---|---|---|---|---|
| rol_app_lectura | Grupo | Solo consulta (lectura) | SELECT sobre catálogos y vistas vigentes |
Necesita visualizar catálogo, productos, pedidos sin modificar datos. Evita permisos de escritura
innecesarios. |
| rol_app_escritura | Grupo | Operaciones transaccionales acotadas | SELECT/INSERT/UPDATE
necesarios para líneas/pedidos; privilegios acotados por objeto | Permite registrar pedidos
manteniendo integridad. Se privilegia uso por SP antes que DML abierto. No incluye
DELETE/TRUNCATE/DDL. |
| rol_soporte | Grupo | Soporte acotado a usuario | SELECT (columnas relevantes) + UPDATE
(columna celular) sobre usuario | Permite atender incidencias sin acceder a campos sensibles
innecesarios ni modificar otros datos. Mínimo indispensable. |
| rol_reportes | Grupo | Solo lectura para reportes | SELECT sobre vistas/resúmenes | Genera
informes sin tocar datos transaccionales. Sin DML. |
| app_web | Login | Consumir aplicación web | Miembro de rol_app_lectura y rol_app_escritura
según necesidad | Login de aplicación, sin SUPERUSER. Sigue principio de mínimo privilegio. |
| admin_datos | Login | Administración acotada con trazabilidad | Privilegios puntuales
necesarios para mantenimiento; NO SUPERUSER | Requiere justificación y auditoría. Se evita
otorgar ALL por defecto. |

## Criterios de aceptación

- Cada GRANT está justificado y acotado al objeto/columna requerido.
- Existe al menos 1 GRANT a nivel de columna sobre usuario.
- ALTER DEFAULT PRIVILEGES definido en esquema public.
- Verificación confirma que cada rol tiene exactamente los privilegios previstos.
- Evidencia registrada en BITACORA.md.
```

ANEXO B - kiro/design.md

```
# Kiro - Design TP U5
```

```
## Objetivo
Diseñar un esquema de seguridad verificable para Food Store: roles/permisos (mínimo privilegio),
auditoría, simulacro de incidente con análisis crítico, anonimización y verificación de
equivalencia.

## Estrategia

### A) Roles y Permisos
1. Crear roles de grupo primero (NOINHERIT donde aplique según criterio).
2. Crear roles de login y asignarlos con GRANT ... TO.
3. Otorgar USAGE sobre schema public.
4. Otorgar privilegios por objeto. Usar privilegios a nivel de columna cuando convenga.
5. Configurar ALTER DEFAULT PRIVILEGES para objetos futuros.
6. Verificar con \du e information_schema.role_table_grants.

### B) Auditoría
1. Activar log_connections, log_disconnections, log_statement='mod' (o nivel que registre
modificaciones).
2. Ejecutar secuencia: lectura, inserción, actualización, intento fallido de login.
3. Capturar fragmento de log.
4. Analizar qué queda registrado y qué quedaría fuera (noción pgaudit).

### C) Simulacro de incidente
1. Ejecutar: intentos fallidos contra fn_autenticar -> login exitoso -> lectura masiva de usuario
-> intento de escalamiento.
2. Verificar si el intento de escalamiento falla. Si no falla -> hallazgo y corrección
obligatoria.
3. Anonimizar log (mails/nombres/PII) antes de enviarlo a IA.
4. Pedir reconstrucción a OpenCode SIN contexto extra.
5. Contrastar: aciertos/omisiones/afirmaciones no sustentadas por log.
6. Decisión de contención tomada por el equipo (documentada).

### D) Anonimización
1. Crear/reutilizar usuario_anon con datos sintéticos deterministas.
2. Verificar equivalencia agregada: conteo por rol, conteo por eliminado. Deben coincidir.

## Medidas de seguridad aplicadas
- Sin SUPERUSER para app_web
- Mínimo privilegio por objeto/columna
- Logs tratados como activo sensible (acceso restringido)
- PII anonimizado antes de compartir con IA
- Validación humana obligatoria ante sugerencias de IA
- Bitácora trazable con al menos un caso de corrección por exceso de privilegio
- Todo ejecutado exclusivamente en entorno de desarrollo
```

ANEXO C - BITACORA.md

```
# BITACORA.md - Uso de IA (OpenCode / Kiro) - TP U5
```

Prompt enviado a la IA	Respuesta relevante recibida (resumida)	Validación o corrección humana aplicada
[OpenCode] "Auditar roles.sql aplicando principio de mínimo privilegio. Buscar privilegios excesivos."	Propuso reemplazar GRANT puntuales por `GRANT ALL ON ALL TABLES IN SCHEMA public` para `rol_soporte`.	**RECHAZADO.** Se mantuvo GRANT a nivel de columna sobre `usuario` (`SELECT` de columnas necesarias + `UPDATE(celular)` únicamente). Se consideró excesivo e inseguro. Registrado como corrección.
[Kiro] "Generar requirements + design para TP U5 Parte A con justificación por rol."	Generó estructura con roles y justificación. Propuso agregar algunos privilegios genéricos.	**AJUSTADO.** Se acotaron privilegios por objeto/columna, se agregó requisito explícito de `ALTER DEFAULT PRIVILEGES` y se reforzó verificación con `information_schema.role_table_grants`.
[OpenCode] "Reconstruir incidente desde este log anonimizado (sin contexto). Proponer hipótesis de contención."	Reconstruyó cronología (fallidos -> exitoso -> lectura masiva -> intento de escalamiento). Propuso medidas: bloquear IP, forzar cambio de contraseñas, revocar rol. Asumió horario/origen con cierta inferencia.	**CONTRASTADO CRÍTICAMENTE.** Se verificó que el log **NO** muestra IP externa distinta ni timestamp absoluto concluyente; se confirmó que el intento de `GRANT admin_datos TO app_web_anon` **falló** (`permission denied`). Se descartaron afirmaciones no sustentadas. Decisión de contención tomada por el equipo con base en evidencia.
[OpenCode] "Generar script para crear usuario_anon"	Propuso copiar valores parcialmente ofuscados o truncar datos reales.	**RECHAZADO.** Se utilizaron datos **sintéticos deterministas** (`md5` + salt + prefijos `NombreAnon/ApellidoAnon`) sin PII real. Contraseña reemplazada por `***ANON***`.

Observación obligatoria: Se documenta explícitamente al menos un caso donde la IA propuso un permiso más amplio de lo necesario (GRANT ALL) y fue **detectado, rechazado y corregido** por validación humana.

ANEXO D - SQL

01_roles_permisos_minimo_privilegio.sql

```
-- ============================================================================
-- TP U5 - PARTE A: Roles y permisos con principio de mínimo privilegio
-- Archivo: 01_roles_permisos_minimo_privilegio.sql
-- ============================================================================

-- 1) Roles de grupo
CREATE ROLE rol_app_lectura NOINHERIT;
CREATE ROLE rol_app_escritura NOINHERIT;
CREATE ROLE rol_soporte NOINHERIT;
CREATE ROLE rol_reportes NOINHERIT;

-- 2) Roles de login
CREATE ROLE app_web LOGIN PASSWORD 'cambiar_en_dev' NOINHERIT;
CREATE ROLE admin_datos LOGIN PASSWORD 'cambiar_en_dev' NOINHERIT;

-- 3) Asignación de roles de grupo a roles de login
GRANT rol_app_lectura TO app_web;
```

```
GRANT rol_app_escritura TO app_web;
GRANT rol_reportes TO app_web;
GRANT rol_soporte TO admin_datos;
GRANT rol_app_lectura TO admin_datos;

-- 4) Permisos a nivel de esquema
GRANT USAGE ON SCHEMA public TO rol_app_lectura, rol_app_escritura, rol_soporte, rol_reportes;

-- 5) Permisos mínimos por objeto
-- Lectura general acotada
GRANT SELECT ON TABLE categoria, producto, pedido, detalle_pedido TO rol_app_lectura;
GRANT SELECT ON TABLE usuario TO rol_app_lectura; -- solo lectura, se restringirá por columna si
aplica

-- Escritura acotada para app
GRANT SELECT, INSERT, UPDATE ON TABLE pedido TO rol_app_escritura;
GRANT SELECT, INSERT, UPDATE ON TABLE detalle_pedido TO rol_app_escritura;
GRANT SELECT ON TABLE producto, categoria, usuario TO rol_app_escritura;

-- Reportes: solo lectura
GRANT SELECT ON TABLE pedido, detalle_pedido, producto, categoria TO rol_reportes;

-- Soporte: permisos a nivel de COLUMNA sobre usuario (mínimo privilegio)
GRANT SELECT (id, nombre, apellido, mail, celular, rol, eliminado, created_at) ON TABLE usuario
TO rol_soporte;
GRANT UPDATE (celular) ON TABLE usuario TO rol_soporte;

-- Admin_datos: permisos acotados (sin SUPERUSER). Ajustar según necesidades verificadas
GRANT USAGE, CREATE ON SCHEMA public TO admin_datos;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO admin_datos;
-- Se recomienda NO otorgar DELETE/TRUNCATE masivo sin justificación

-- 6) ALTER DEFAULT PRIVILEGES para heredar política en objetos futuros
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO rol_app_lectura;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE ON TABLES TO rol_app_escritura;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO rol_reportes;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT (id, nombre, apellido, mail, celular, rol, eliminado, created_at), UPDATE
(celular) ON TABLES TO rol_soporte;

-- 7) Verificación
-- \du
-- SELECT grantee, table_schema, table_name, privilege_type, column_name
-- FROM information_schema.role_table_grants
-- WHERE grantee IN
```

```
('rol_app_lectura','rol_app_escritura','rol_soporte','rol_reportes','app_web','admin_datos')
-- ORDER BY grantee, table_name, privilege_type;
```

02_auditoria_configuracion_y_pruebas.sql

```
-- ============================================================================
-- TP U5 - PARTE B: Activación y prueba de auditoría
-- Archivo: 02_auditoria_configuracion_y_pruebas.sql
-- ============================================================================

-- 1) CONFIGURACIÓN DEL SERVIDOR (documentar en informe)
-- En postgresql.conf o con ALTER SYSTEM (entorno de desarrollo):
-- ALTER SYSTEM SET log_connections = 'on';
-- ALTER SYSTEM SET log_disconnections = 'on';
-- ALTER SYSTEM SET log_statement = 'mod';  -- registra DDL/DML
-- SELECT pg_reload_conf();

-- Valores mínimos requeridos:
-- log_connections = on
-- log_disconnections = on
-- log_statement = mod (al menos registra operaciones de modificación)

-- 2) SECUENCIA DE PRUEBAS
-- Conectar con distintos roles/sesiones y ejecutar:

-- a) Lectura
SELECT count(*) FROM producto WHERE eliminado = FALSE;

-- b) Inserción (ejemplo controlado - ajustar a datos válidos)
-- BEGIN;
-- INSERT INTO categoria(nombre, descripcion) VALUES ('CAT_AUDIT_TEST','PRUEBA AUDITORIA') ON
CONFLICT DO NOTHING;
-- COMMIT;

-- c) Actualización
-- BEGIN;
-- UPDATE producto SET descripcion = COALESCE(descripcion,'') WHERE id = (SELECT id FROM producto
LIMIT 1);
-- COMMIT;

-- d) Intento de inicio de sesión fallido
-- Simular desde cliente/psql con credenciales incorrectas contra fn_autenticar o conexión
directa.
-- Registrar fragmento de log resultante en logs/fragmento_log_pruebas_auditoria.txt

-- 3) ANÁLISIS (para informe)
-- Útil para auditoría: quién (usuario/rol), qué (sentencia), cuándo (timestamp), desde dónde
(host/puerto), conexión/desconexión.
-- Fuera de alcance nativo (noción): detalle fila por fila con valores antiguos/nuevos, usuario
```

```
que realmente ejecutó acción
-- heredada, trazabilidad por objeto/columna con contexto completo -> pgaudit permite mayor
granularidad.
```

03_simulacro_incidente_acceso_indebido.sql

```
-- ============================================================================
-- TP U5 - PARTE C: Simulacro de incidente de acceso indebido (SOLO DESARROLLO)
-- Archivo: 03_simulacro_incidente_acceso_indebido.sql
-- ============================================================================

-- IMPORTANTE: Ejecutar ÚNICAMENTE contra instancia propia de Food Store en desarrollo.
-- NO probar contra sistemas de terceros.

-- SECUENCIA REQUERIDA
-- 1) Varios intentos fallidos de inicio de sesión contra fn_autenticar
-- Ejemplo:
-- SELECT fn_autenticar('noexiste@foodstore.test', 'clave_mala') AS r1;
-- SELECT fn_autenticar('noexiste2@foodstore.test', 'clave_mala') AS r2;
-- SELECT fn_autenticar('noexiste3@foodstore.test', 'clave_mala') AS r3;

-- 2) Inicio de sesión exitoso
-- SELECT fn_autenticar('usuario_valido@foodstore.test', 'clave_correcta') AS ok;

-- 3) Lectura masiva de la tabla usuario
-- SELECT * FROM usuario;  -- o COUNT(*) + SELECT con límite para evidencia

-- 4) Intento de otorgarle a un rol de bajo privilegio membresía en admin_datos
-- (o ejecutar operación reservada a admin_datos)
-- Con rol de bajo privilegio:
-- BEGIN;
-- GRANT admin_datos TO <rol_bajo_privilegio>;  -- DEBE FALLAR si esquema correcto
-- ROLLBACK;

-- VERIFICACIÓN
-- Si el GRANT anterior NO falla -> HALLAZGO: esquema de permisos tiene defecto. Corregir Parte A
antes de continuar.
-- Documentar resultado en informe de contención.

-- ANONIMIZACIÓN PREVIA A IA
-- Extraer fragmento de log correspondiente a toda la secuencia
-- Reemplazar mails, nombres, hosts/sesiones identificables por valores sintéticos
-- Guardar en logs/log_incidente_anonimizado.txt
-- Enviar SOLO este log anonimizado a OpenCode (sin contexto extra)
-- Contrastar respuesta IA vs ejecución real y documentar en BITACORA.md + informe
```

04_usuario_anon_anonimizacion.sql

```
-- ============================================================================
-- TP U5 - PARTE D: Anonimización de datos personales
-- Archivo: 04_usuario_anon_anonimizacion.sql
-- ============================================================================

-- Crear tabla usuario_anon con datos sintéticos deterministas
DROP TABLE IF EXISTS usuario_anon CASCADE;

CREATE TABLE usuario_anon AS
SELECT
  id,
  ('NombreAnon' || id) AS nombre,
  ('ApellidoAnon' || id) AS apellido,
  (md5(id::text || 'foodstore_u5_salt') || '@anon.foodstore.test') AS mail,
  ('11' || lpad((abs(hashtextextended(id::text, 12345)) % 90000000)::text, 8, '0')) AS celular,
  '***ANON***' AS contrasena,
  rol,
  eliminado,
  created_at
FROM usuario;

-- Índices opcionales para consultas
CREATE INDEX IF NOT EXISTS idx_usuario_anon_rol ON usuario_anon(rol);
CREATE INDEX IF NOT EXISTS idx_usuario_anon_eliminado ON usuario_anon(eliminado);

-- Verificación básica
SELECT count(*) AS total_usuario FROM usuario;
SELECT count(*) AS total_usuario_anon FROM usuario_anon;
SELECT count(*) AS diff_total FROM (SELECT id FROM usuario EXCEPT SELECT id FROM usuario_anon) x;
```

05_verificacion_equivalencia_anonimizacion.sql

```
-- ============================================================================
-- TP U5 - PARTE D: Verificación de equivalencia tras anonimización
-- Archivo: 05_verificacion_equivalencia_anonimizacion.sql
-- ============================================================================

-- Conteo por rol: deben coincidir exactamente
SELECT rol, count(*) AS cantidad
FROM usuario
GROUP BY rol
ORDER BY rol;

SELECT rol, count(*) AS cantidad
FROM usuario_anon
GROUP BY rol
ORDER BY rol;

-- Distribución por eliminado: deben coincidir exactamente
```

```
SELECT eliminado, count(*) AS cantidad
FROM usuario
GROUP BY eliminado
ORDER BY eliminado;

SELECT eliminado, count(*) AS cantidad
FROM usuario_anon
GROUP BY eliminado
ORDER BY eliminado;

-- Totales: deben coincidir
SELECT (SELECT count(*) FROM usuario) AS total_original,
       (SELECT count(*) FROM usuario_anon) AS total_anon,
       (SELECT count(*) FROM usuario) = (SELECT count(*) FROM usuario_anon) AS totales_iguales;
```

ANEXO E - Logs

fragmento_log_pruebas_auditoria.txt

```
# FRAGMENTO DE LOG - PRUEBAS DE AUDITORÍA (PARTE B)
# ANONIMIZADO - ENTORNO DE DESARROLLO

[YYYY-MM-DD HH:MM:SS TZ] LOG: connection authorized: user=[USUARIO_PRUEBA] database=food_store
host=[IP_DEV]
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT count(*) FROM producto WHERE eliminado = FALSE;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: INSERT INTO categoria(nombre,descripcion) VALUES
('CAT_AUDIT_TEST','PRUEBA AUDITORIA') ON CONFLICT DO NOTHING;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: UPDATE producto SET descripcion =
COALESCE(descripcion,'') WHERE id = <ID_PRODUCTO>;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  connection authorized: user=[USUARIO_FALLIDO] database=food_store
host=[IP_DEV]
[YYYY-MM-DD HH:MM:SS TZ] FATAL:  password authentication failed for user "[USUARIO_FALLIDO]"
[YYYY-MM-DD HH:MM:SS TZ] LOG: disconnection: session time: <TIEMPO> user=[USUARIO_PRUEBA]
database=food_store host=[IP_DEV]
```

log_incidente_anonimizado.txt

```
# LOG ANONIMIZADO - SIMULACRO DE INCIDENTE (PARTE C)
# ENVIADO A OpenCode SIN CONTEXTO EXTRA

[YYYY-MM-DD HH:MM:SS TZ] LOG: connection authorized: user=app_web_anon database=food_store
host=127.0.0.1
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT fn_autenticar('anon_user_01@anon.test','***') AS
r;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT fn_autenticar('anon_user_02@anon.test','***') AS
r;
```

```
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT fn_autenticar('anon_user_03@anon.test','***') AS
r;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT fn_autenticar('app_web_anon@anon.test','***') AS
r;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: SELECT * FROM usuario;
[YYYY-MM-DD HH:MM:SS TZ] LOG:  statement: GRANT admin_datos TO app_web_anon;
[YYYY-MM-DD HH:MM:SS TZ] ERROR:  permission denied for role admin_datos
[YYYY-MM-DD HH:MM:SS TZ] LOG: disconnection: session time: <TIEMPO> user=app_web_anon
database=food_store host=127.0.0.1
```